

# Activity 7 - Apply stock movements with enums and match

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K3/A3

## Specification

Movement::{Receive(u32), Ship(u32), Adjust(i64)}. apply(stock, movement) -> Result<u32, StockError>: shipping more than available is Insufficient; results below zero are NegativeResult; above u32 are Overflow. apply\_all stops at the first error.

## Part 1: Learn Rust prompts

### Explain it simply

I am learning Rust. Explain enums with data, match with guards, if let, ranges in patterns and loops to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

### Walk me through the sample

Here is samples/01\_enum\_match.rs from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

### Quiz me

Quiz me on enums with data, match with guards, if let, ranges in patterns and loops with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

### Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles control flow: enum, match, if and loops with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

## Part 2: Build with AI (vibe-coding prompts)

### Plan before code

Act as a senior Rust reviewer. Specification: Movement::{Receive(u32), Ship(u32), Adjust(i64)}. apply(stock, movement) -> Result<u32, StockError>: shipping more than available is Insufficient; results below zero are NegativeResult; above u32 are Overflow. apply\_all stops at the first error. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

### Generate with AI

Implement "Apply stock movements with enums and match" in starter/src so that the supplied tests pass. Use enum + match with exhaustive arms; no wildcard \_ arm that hides future variants. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

## Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

## Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

## Part 3: Rust sample scripts (samples/ folder)

### 01\_enum\_match.rs

Run: rustc --edition 2021 01\_enum\_match.rs && ./01\_enum\_match

// Sample: an enum with data and an exhaustive match with a guard.

```
enum Movement {
    Receive(u32),
    Ship(u32),
    Adjust(i64),
}

fn describe(movement: &Movement) -> String {
    match movement {
        Movement::Receive(qty) => format!("receive {qty}"),
        Movement::Ship(qty) if *qty > 100 => {
            format!("ship {qty} (large order, needs approval)")
        }
        Movement::Ship(qty) => format!("ship {qty}"),
        Movement::Adjust(delta) => format!("adjust by {delta:+}"),
    }
}

fn main() {
    let moves = [
        Movement::Receive(10),
        Movement::Ship(3),
        Movement::Ship(250),
        Movement::Adjust(-2),
    ];
    for movement in &moves {
        println!("{}", describe(movement));
    }
}
```

Expected output:

```
receive 10
ship 3
ship 250 (large order, needs approval)
adjust by -2
```

### 02\_ranges\_if\_let\_loops.rs

Run: rustc --edition 2021 02\_ranges\_if\_let\_loops.rs && ./02\_ranges\_if\_let\_loops

// Sample: match on ranges, if let, and a while loop.

```
fn status(qty: u32) -> &'static str {
    match qty {
        0 => "out of stock",
        1..=4 => "low",
        5..=99 => "ok",
        _ => "bulk",
    }
}

fn main() {
    for qty in [0, 3, 5, 99, 100] {
        println!("{qty:>3} -> {}", status(qty));
    }
}
```

```

    }
    let found: Option<u32> = Some(7);
    if let Some(qty) = found {
        println!("if let found {qty}");
    }
    let mut countdown = 3;
    while countdown > 0 {
        print!("{countdown} ");
        countdown -= 1;
    }
    println!("go");
}

```

Expected output:

```

0 -> out of stock
3 -> low
5 -> ok
99 -> ok
100 -> bulk
if let found 7
3 2 1 go

```

## Learn more

Rust Book: Defining an Enum: <https://doc.rust-lang.org/book/ch06-01-defining-an-enum.html>

Rust Book: The match Control Flow Construct: <https://doc.rust-lang.org/book/ch06-02-match.html>

W3Schools: Rust Enums: [https://www.w3schools.com/rust/rust\\_enums.php](https://www.w3schools.com/rust/rust_enums.php)

W3Schools: Rust Match: [https://www.w3schools.com/rust/rust\\_match.php](https://www.w3schools.com/rust/rust_match.php)

W3Schools: Rust If .. Else: [https://www.w3schools.com/rust/rust\\_if\\_else.php](https://www.w3schools.com/rust/rust_if_else.php)

W3Schools: Rust Loops: [https://www.w3schools.com/rust/rust\\_loops.php](https://www.w3schools.com/rust/rust_loops.php)

Programiz: Rust Enum: <https://www.programiz.com/rust/enum>

Programiz: Rust Loop: <https://www.programiz.com/rust/loop>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: [https://www.w3schools.com/rust/rust\\_getstarted.php](https://www.w3schools.com/rust/rust_getstarted.php)